

Retrospectiva 1

1. ¿Cuáles fueron los mini-ciclos definidos? Justifiquenlos.

Planificación en mini-ciclos Nuestro enfoque en la formación de nuestros ciclos es avanzar a partir de lo que no depende de nada hacia lo que depende de todo, así tendremos en cuenta que cada uno de los mini-ciclos entregue algo que sea ejecutable y además verificable en BlueJ antes de pasar a realizar el siguiente. También decidimos este orden porque somos conscientes de nuestro nivel de programación y de diseño, entonces decidimos que primero funcione lo básico y solo cuando esta parte sea comprendida por ambos abordaremos la complejidad del problema, como hacerlo robusto.

Mini-ciclo 1 El esqueleto:

Objetivo: Crear las clases del proyecto y lograr que compile. Definiremos la estructura base sin la lógica como tal, la clase principal y sus colaboradoras existen, también que el paquete shapes quede integrado. Es nuestro principal mini-ciclo porque no depende de nada, ninguna operación puede funcionar si no realizamos las clases. Los métodos que abarcaremos será solo el constructor de slotMachine y las clases Wheel y Symbol creadas.

Mini-ciclo 2 Gestionar elementos:

Objetivo: Dar contenido a la maquina como tal, es decir, vamos a agregar y eliminar ruedas, y agregar y eliminar símbolos. La máquina ya puede llenarse, aunque todavía no gira ni se ve. Los métodos que usaremos será addWheel , delWheel, addSymbol, delSymbol, symbols y ok. En este mini-ciclo si tenemos una dependencia y es que depende del anterior ya que las clases deben existir para poder crear y almacenar que agreguemos o queramos eliminar.

Mini - ciclo 3 Dinamismo:

Objetivo: La razón de ser de simular la maquina poner en movimiento la máquina, girando las ruedas, forzando un símbolo en una rueda y consultado el estado. Usaremos los métodos spin, spin(Wheel), placeSymbol, configuration, distinctSymbols e isJacktpot. Este ciclo depende del mini-ciclo anterior, ya que gracias a este podremos agregar ruedas y símbolos para poder girarlos y consultarlos.

Mini - ciclo 4 Visual:

Objetivo: Hacer que la maquina se vea, integraremos la representación gráfica sobre el modelo ya funcional y extenderemos a Canvas para que reconozca los colores del estándar CSS como es solicitado en los requisitos. Usaremos makeVisible, makeInvisible, la extensión de Canvas

para los colores y el aspecto que debe tener la máquina para verse de manera distinta al estar en un estado ganador.

Mini - ciclo 5 Robustez:

Objetivo: Revisar casos límites y errores que pueden llegar a pasar, en este mini- ciclo rechazaremos entradas que no son validas, este también abarcara la validación de color CSS, el rechazo de un color repetido, las posiciones que estén fuera de rango, cuando ok = false y los mensajes con JOptionPane, que se mostraran únicamente si la maquina se encuentra visible. Este depende de todos los mini- ciclos depende de todos los anteriores.

Mini - ciclo 6 Cierre:

Objetivo: Terminar nuestro simulador de maquina tragamonedas con un cierre, utilizando el método exit, que finalizara el simulador. Este depende de todos los mini ciclos anteriores.

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?

En términos de implementación no hemos iniciado ningún mini-ciclo ya que no alcanzamos a escribir código. Lo que completamos fue la etapa previa de diseño, que considerábamos algo indispensable antes de programar, además que tener esto muy bien hecho nos sirvió a imaginar como implementaríamos de manera correcta el código. La razón de este estado es que esta primera entrega fue muy difícil para nosotros como equipo que apenas comienza, nos tomó la mayor parte del tiempo entender cómo usar Astah, como definir de buena forma el diagrama de clases y comprender que realmente estaban bien los diagramas de secuencia.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Nombre)

Juan Sebastián Garavito Mejia: 10 horas

Laila Zareth Romano Guerrero: 12 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue poder construir el diagrama de clases que a nuestro parecer esta de forma coherente con el enunciado, entendimos las decisiones del diseño detrás del diagrama y no solo creamos clases porque si, como pensábamos que era al principio un diagrama de clases. Lo consideramos el nuestro mayor logro porque el diagrama nos permitió entender que la maquina tragamonedas debe respetar la regla de que todas las ruedas tienen los mismos símbolos en el mismo orden y entender todo esto nos costó mucho, pero nos hizo tener claridad sobre todo el diseño, aunque sigue siendo algo muy abstracto primero diseñar sin tener código y frustra, pero a la final aprendemos.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El manejo de Astah, al ser algo nuevo para nosotros nos costó mucho entender cómo poner los parámetros y los tipos que van en campos separados del nombre, y que Astah genera clases automáticamente al escribir tipos como ArrayList, muchas veces lo borramos y no entendíamos porque se borrada, pero resolvimos aprendiendo a usar las propiedades de la herramienta consultando tutoriales en YouTube, también nos costó mucho separar de que es lo que realmente va en el diagrama y que no, sus relaciones y nombres de estas no las entendíamos para nada, pero pudimos.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Siendo honestos, esta primera entrega fue difícil para nosotros, como equipo logramos comprender el problema, definir los mini-ciclos y construir el diagrama de clases y los diagramas de secuencia, pero no alcanzamos a implementar código. Nos costó entender cómo usar Astah, como definir de buena forma el diagrama de clases y comprender los diagramas de secuencia y porque se estructuran de esa manera. También nos tomó tiempo entender con claridad que era lo que teníamos que hacer y ponernos de acuerdo como equipo. Lo que hicimos bien fue realmente no rendirnos ante esas dificultades e insistimos hasta comprender las decisiones de diseño en lugar de dibujar sin entender y terminamos (desde nuestro punto de vista) con un diseño que si refleja el problema. Nos comprometemos a ser mucho más constantes en el aprendizaje de las c nuevas herramientas, a dedicarles tiempo con anticipación y no sobre la marcha y a enfocarnos en el diseño sin pensar en el código, entendiendo que primero debemos modelar y después si podemos implementar.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

La práctica XP más útil para nosotros fue el juego de la planificación, fue la que nos permitió dividir un problema que al inicio ninguno de los dos entendía, que nos parecía grande y difícil de visualizar de manera real. Entonces en lugar de intentar abordar todo el simulador de la maquina tragamonedas, planificamos incrementos que van desde lo más simple a lo más complejo como se puede ver en nuestros mini-ciclos y esto lo consideramos lo más valioso porque le dio forma a nuestro trabajo y nos permitió avanzar de manera un poco más clara.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

Referencias:

- IS-FUNDAMENTOS. (s.f.). Unidad 1: Control de Versiones (CVS). Monitoria de DOPO y MYSD, Escuela Colombiana de Ingeniería Julio Garavito
<https://fis-eci.github.io/software-foundations-eci/Unidad-CVS/>

- [Lucid Software Español]. [UML Tutorial: Diagrama de Clases desde cero] YouTube <https://www.youtube.com/watch?v=Z0yLerU0g-Q>

La más útil fue el video de YouTube sobre el diagrama de clases ya que para nosotros fue confuso saber qué relaciones y que tipo de relación debíamos tener en nuestro diagrama de clases y el video fue muy claro para nosotros entender esto.

Retrospectiva Ciclo 2

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos

Mini ciclo 7 Extensión de ruedas manage wheels:

Objetivo: intercambiar dos ruedas y fijar /soltar una rueda. Los métodos que usaremos son swap, lock, unlock y en Wheel el estado locked.

Mini ciclo 8 Extencion de giro spin:

Objetivo: rotar una rueda un numero de pasos y dejar la maquina en una configuración dada. Los métodos que usaremos son spin(Wheel, steps) y spin(setSymbols)

Mini ciclo G Pruebas:

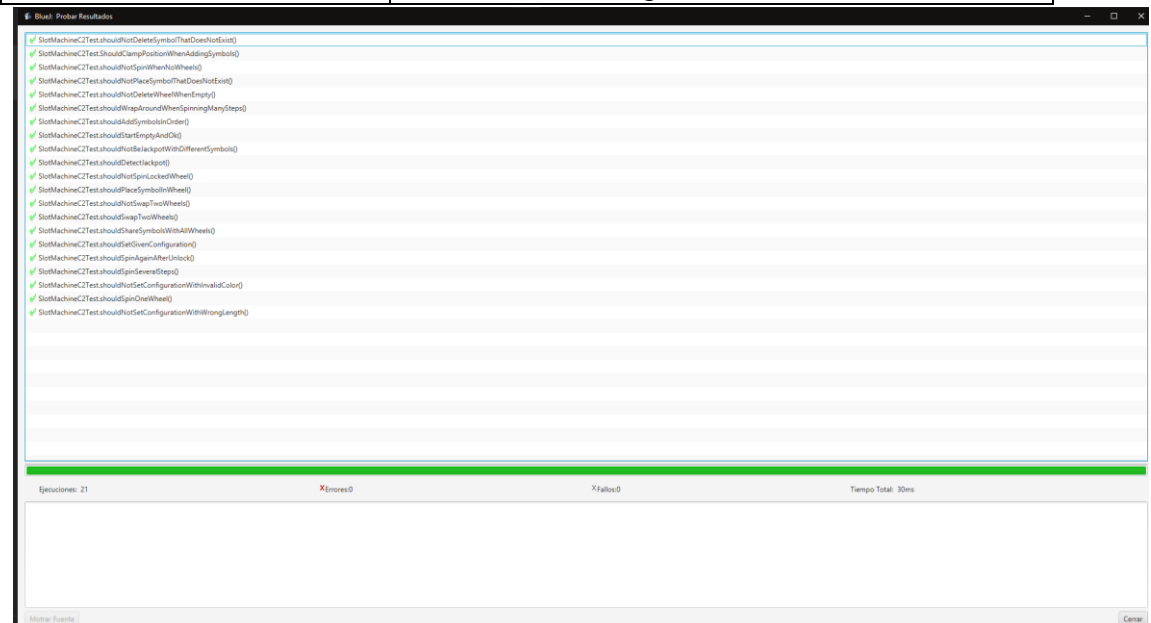
Objetivos: verificar todo lo construido con pruebas en modo invisible.

2. Pruebas de unidad

Mini ciclo	Tests
1 y 2	shouldStartEmptyAndOk, shouldAddSymbolsInOrder, shouldClampPositionWhenAddingSymbols, shouldNotDeleteSymbolThatDoesNotExist, shouldShareSymbolsWithAllWheels, shouldNotDeleteWheelWhenEmpty shouldStartEmptyAndOk,
3	shouldSpinOneWheel, shouldNotSpinWhenNoWheels, shouldPlaceSymbolInWheel, shouldNotPlaceSymbolThatDoesNotExist, shouldDetectJackpot, shouldNotBeJackpotWithDifferentSymbols
7	shouldSwapTwoWheels, shouldNotSwapWhenNoWheels, shouldNotSpinLockedWheel, shouldSpinAgainAfterUnlock

8

shouldSpinSeveralSteps,
shouldWrapAroundWhenSpinningManySteps,
shouldSetGivenConfiguration,
shouldNotSetConfigurationWithWrongLength,
shouldNotSetConfigurationWithInvalidColor



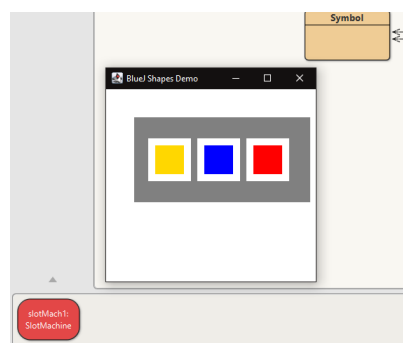
Acá se demuestra que todos los test se hicieron exitosamente.

3. Pruebas de unidad

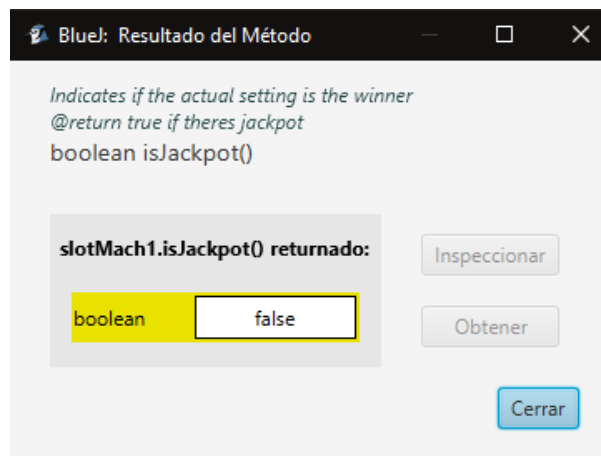
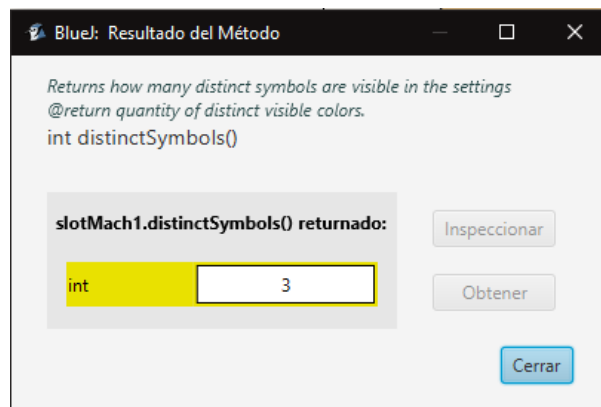
- **Prueba 1:** Verificar que un usuario puede construir la máquina, girarla, consultar su estado y llevarla a una configuración para hacer Jackpot.

Paso 1 al 3: lo que se hizo en estos pasos fue crear una maquina las ruedas y los “símbolos” que iba a tener. En este caso se crearon 3 ruedas con los símbolos “red”, “gold” y “blue” en ese orden.

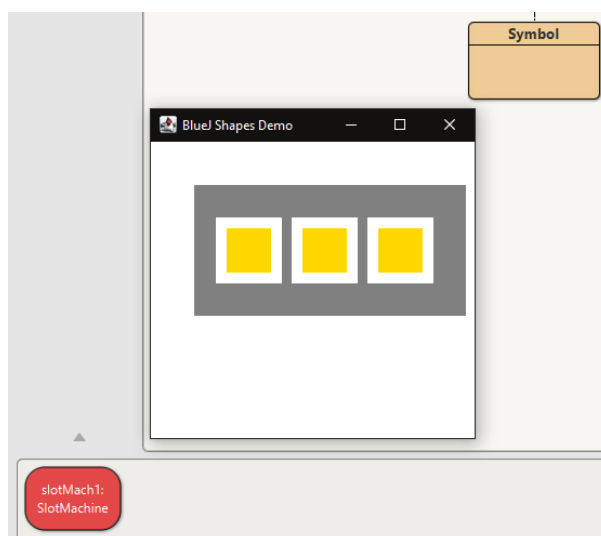
Paso 4 y 5: En este paso se hacen girar las ruedas número 1 y 2 para quedar en los colores respectivos y hacer visible la máquina.



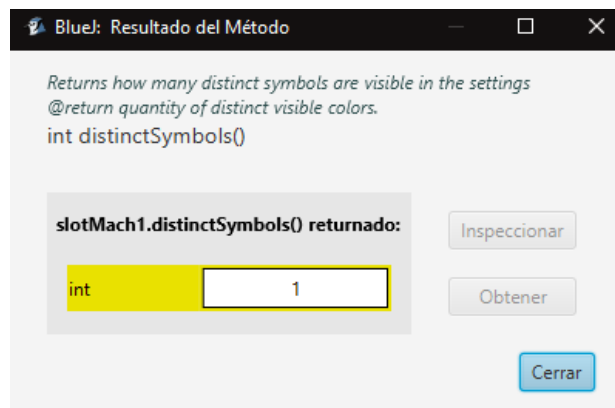
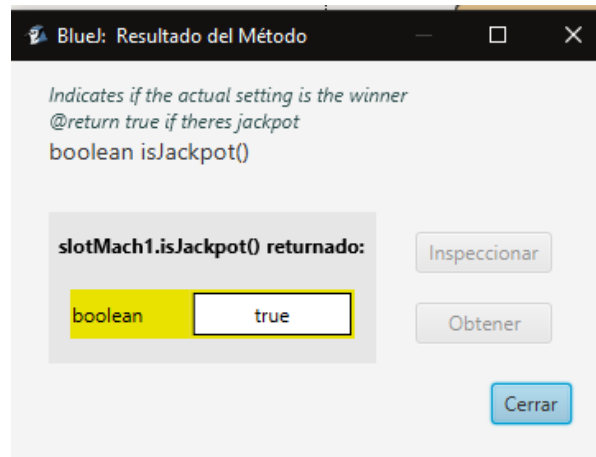
Paso 6: Se verifica que efectivamente hay 3 símbolos distintos siendo mostrados y se verifica que `isJackpot() = false` ya que no están las tres ruedas con el mismo símbolo.



Paso 7: Se giran mediante `spin(new String[])` para que todas queden el símbolo "gold".



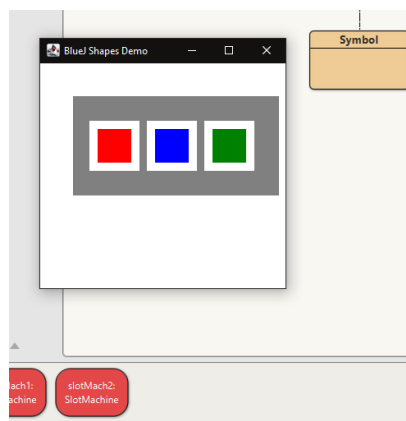
Paso 8 Verificar que `isJackpot()` = true ya que todas las ruedas tienen el mismo símbolo (“gold”) y verificar que solo hay 1 símbolo en las ruedas.



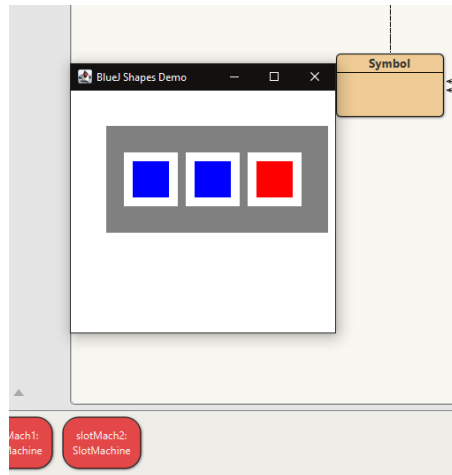
- **Prueba 2:** Verificar que las funciones del ciclo 2 funcionen, fijar una rueda, girar sin que las fija se mueva, intercambiar ruedas y rotar una rueda varios pasos.

Paso 1: se creó la maquina con tres ruedas y tres símbolos “red”, “blue” y “green” en ese orden

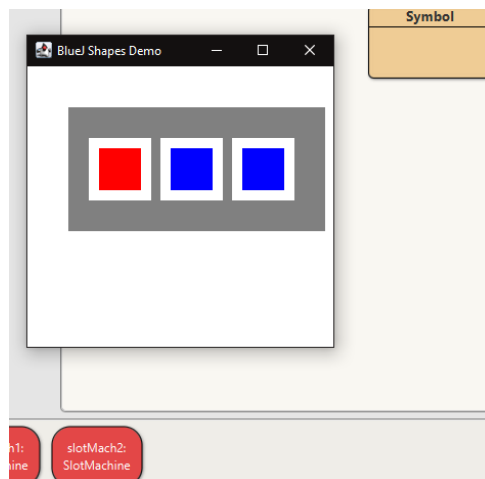
Paso 2: Hacer visible la maquina con las ruedas en el orden mencionado anteriormente.



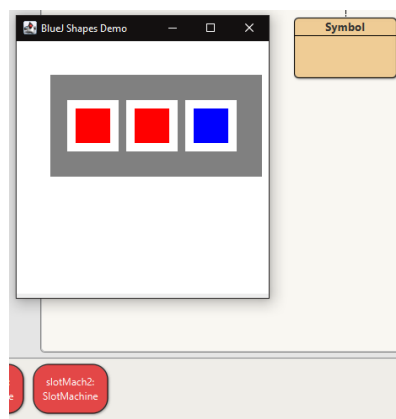
Paso 3 y 4: Primero se bloquea la rueda 2 y seguidamente mediante la función `spin()` se hace girar todas las ruedas, observando que giran las ruedas 1 y 3 pero la 2 no ya que está bloqueada.



Paso 5 y 6: Se desbloquea la rueda 2 mediante `unlock()` y se intercambian las ruedas obteniendo el resultado de la imagen.



Paso 7: En este último paso se hace girar la rueda dos 5 veces obteniendo el símbolo "red"



**2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos?
¿por qué?**

A diferencia de nuestra retrospectiva del ciclo anterior, en el cual solo alcanzamos la etapa de diseño, en el ciclo 2 logramos implementar código. Nuestros mini-ciclos 1 Esqueleto, 2 Gestión de elementos, 3 Dinamismo y 6 Cierre, quedaron en un estado completo. El mini-ciclo 4 Visual quedó casi completo; la máquina ya se hace visible e invisible y extendimos Canvas para reconocer los colores del estándar CSS, pero nos faltó el detalle de que la máquina al llegar un estado ganador esta luzca de manera diferente. El mini-ciclo 5 Robustez quedó a medias, la saturación de posición que estén fuera del rango, el manejo de que ok() sea false y los mensajes con JOptionPane que solo aparecen si la máquina es visible, pero aun nos falta validar que el color elegido sea realmente un color CSS y rechazar colores repetidos.

Para nuestra entrega del ciclo 2 definimos 3 mini-ciclos nuevos, que corresponden a la extensión de este; el mini-ciclo 7 que es la extensión de ruedas (swap(), lock(), unlock()) y el mini-ciclo 8 que es la extensión de giro (spin(Wheel,steps), spin(setSymbols)), cada uno de estos ciclos se encuentran completados y el mini-ciclo 9 Pruebas, en el que completamos las pruebas de unidad en modo invisible y las dos pruebas de aceptación, quedando pendientes realizar las pruebas compartidas del curso.

Nuestra razón principal por la que no hemos terminado unos ciclos que definimos en la entrega uno, es porque decidimos avanzar de lo que no depende de nada hacía lo que depende de todo, dejando la robustez y lo visual al final porque son los que dependen de que el resto funcione.

**3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes?
(Horas/Hombre)**

Juan Sebastián Garavito Mejía: 18 horas

Laila Zareth Romano Guerrero: 14 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue lograr que el simulador funcionara de verdad en código, no solo en diseño como en el ciclo pasado. Pudimos implementar toda la lógica del Ciclo 1 y además las extensiones del Ciclo 2 (intercambiar ruedas, fijar y soltar una rueda, girar un número de pasos y dejar la máquina en una configuración dada). Lo consideramos el mayor logro porque partimos de una entrega anterior donde no habíamos escrito nada de código, y esta vez no solo escribimos las

clases, sino que las probamos y comprobamos que respetan las decisiones de diseño que habíamos tomado, en especial que cada rueda tenga sus propios símbolos para que un mismo color pueda verse en varias ruedas a la vez. Ver la máquina girando y llegando a un jackpot nos confirmó que el diseño que tanto nos costó entender en el ciclo anterior sí era correcto.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue una decisión de diseño: cómo hacer que un mismo símbolo pudiera mostrarse en varias ruedas al mismo tiempo. Al principio teníamos los símbolos compartidos por todas las ruedas, pero eso chocaba con la idea de que el símbolo tuviera su propio dibujo, porque un solo rectángulo no puede estar en dos lugares del canvas a la vez. Lo resolvimos cambiando el diseño para que cada rueda tuviera sus propios objetos símbolo (pasando de una agregación a una composición), de modo que la máquina replica cada cambio de símbolo en todas las ruedas para mantenerlas iguales. También tuvimos que aprender a manejar el rectángulo del paquete shapes, que solo se mueve por desplazamientos relativos y no absolutos, así que llevamos por dentro la cuenta de la posición de cada figura. Resolverlo nos obligó a entender bien la diferencia entre datos compartidos y datos propios, y las multiplicidades en el diagrama de clases.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Como equipo hicimos bien el mantener el orden de los mini-ciclos: no intentamos hacer todo de golpe, sino que avanzamos método por método, comprobando que cada parte funcionara antes de seguir con la siguiente. También mejoramos respecto al ciclo anterior en que esta vez sí llegamos a la implementación y a las pruebas, no solo al diseño. Nos comprometemos a no dejar la robustez y lo visual para el final del todo, porque son las partes que quedaron incompletas, y a repartirnos mejor el trabajo de las pruebas para alcanzar a hacer también las compartidas del curso con anticipación y no sobre la marcha.

7. Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?

En este ciclo la práctica XP más útil fueron las pruebas de unidad (*testing*). En el ciclo pasado la más útil había sido el juego de la planificación, pero ahora que ya teníamos código, escribir pruebas nos obligó a pensar cada método desde dos preguntas: qué debería hacer y qué no debería hacer. Eso nos ayudó a encontrar detalles que se nos habían pasado y a ganar confianza en que los métodos, incluidos los

nuevos del Ciclo 2, se comportan bien tanto en los casos válidos como en los inválidos. Además, correr las pruebas en modo invisible nos demostró por qué era importante que la lógica funcionara sin depender de la parte gráfica.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

En este ciclo no recurrimos a referencias externas nuevas. El trabajo se apoyó principalmente en el diseño y la comprensión que habíamos construido en el Ciclo 1, y en el material de los laboratorios de la asignatura. Al ser un ciclo de refactorización y extensión sobre una base que ya conocíamos, no necesitamos consultar fuentes adicionales como sí ocurrió en la primera entrega.

Retrospectiva ciclo 3

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos

Mini-ciclo 10 Constructor aleatorio:

Objetivo: crear una máquina de n ruedas y n símbolos inicializada al azar, con el constructor `SlotMachine(n)`, y hacer que `spin()` (todas las ruedas) gire una cantidad aleatoria como una tragamonedas real. Es la base porque el resolovedor necesita una máquina de tamaño n lista para trabajar. No depende del algoritmo.

Mini-ciclo 11 Resolver:

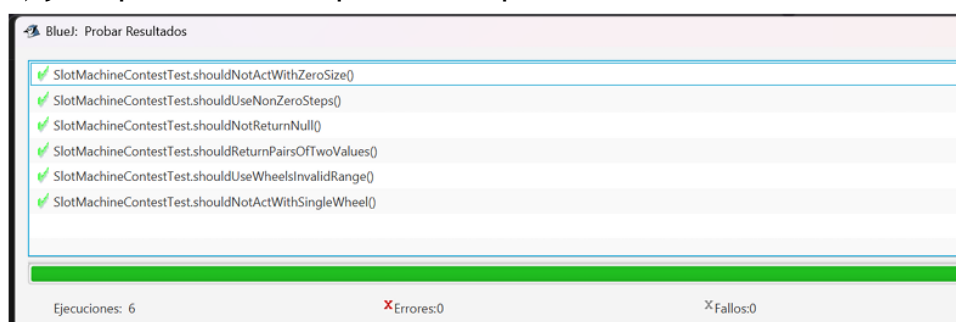
Objetivo: crear la clase `SlotMachineContest` con el método `solve(n)`, que resuelve la máquina en modo invisible usando únicamente `distinctSymbols()` y `spin(wheel, steps)` (la máquina como herramienta de prueba). Es el corazón del ciclo. Depende del mini-ciclo 10.

Mini-ciclo 12 Simular:

Objetivo: `simulate(n)`, que hace lo mismo que `solve` pero con la máquina visible, para ver el proceso paso a paso. Depende del mini-ciclo 11.

Mini-ciclo 13 Pruebas:

Objetivo: verificar lo construido con `SlotMachineContestTest` (modo invisible) y las pruebas de aceptación. Depende de todos los anteriores.



2. ¿Cuál es el estado actual del proyecto en términos de mini ciclos? ¿Por qué?

El proyecto está en su estado más completo. Los mini-ciclos 1 (Esqueleto), 2 (Gestión de elementos), 3 (Dinamismo) y 6 (Cierre) siguen completos. El mini-ciclo 4 (Visual), que en el ciclo pasado quedó casi completo, ahora quedó completo: agregamos el aspecto ganador (jackpot dorado). El mini-ciclo 5 (Robustez) sigue parcial: tenemos la saturación de posiciones, el manejo de ok() en false y los mensajes con JOptionPane solo si la máquina es visible, pero aún falta validar que el color sea realmente un color CSS y rechazar colores repetidos. Los mini-ciclos 7 y 8 (extensiones del Ciclo 2) siguen completos.

De los mini-ciclos nuevos, el 10 (Constructor aleatorio), el 11 (Resolver) y el 12 (Simular) quedaron completos. El mini-ciclo 13 (Pruebas) quedó con las pruebas de unidad propias y las de aceptación completas, y pendiente las pruebas compartidas del curso. La razón de este estado es que priorizamos lo que no depende de nada (el constructor) antes que el algoritmo, y dejamos las pruebas al final porque no se puede probar lo que aún no existe.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Nombre)

Juan Sebastian Garavito Mejia: 14 horas

Laila Zareth Romano Guerrero: 17 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue resolver el problema de la maratón a ciegas: hacer que la máquina llegue al jackpot sin poder ver los símbolos, guiándonos solo por el número de símbolos distintos. Lo consideramos el mayor logro porque es un problema de la final mundial del ICPC y, para un equipo que apenas comienza, entender que había que usar la máquina como herramienta de prueba fue un salto grande. Ver el simulador resolverse solo hasta el jackpot dorado nos confirmó que todo el diseño de los ciclos anteriores estaba bien hecho. Aunque somos conscientes que nos toca mejorar la robustez, tener una mayor estética de la maquina y una mejor visualización de la maquina en caso tal de que el usuario quisiera poner el máximo de ruedas permitidas.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue el algoritmo de solve. Primero intentamos lo más natural: girar cada rueda a la posición con menos símbolos distintos y repetir. Lo probamos y no funcionaba: se quedaba atascado y

no llegaba al jackpot. Entendimos la razón de fondo: el número de distintos no dice a cuál rueda te estás uniendo, así que la estrategia simple cae en un mínimo local (por ejemplo, con la mitad de un color y la mitad de otro, ningún movimiento de una sola rueda mejora). Lo resolvimos usando el movimiento de la rueda 1 como referencia: primero la llevamos a un símbolo único, luego, para cada rueda, la giramos una vuelta con la rueda 1 quieta y otra con la rueda 1 movida un paso, y la posición que reacciona a ese movimiento es la que iguala a la rueda 1. Anotamos todas las rotaciones y las aplicamos al final. Descubrir esto nos obligó a entender por qué la información disponible (solo un número) limita lo que se puede hacer.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Hicimos bien en no conformarnos con la primera solución, al ver que la estrategia simple fallaba, la medimos, entendimos por qué, y buscamos una correcta en vez de entregar algo que no servía. También cerramos pendientes de ciclos anteriores (el aspecto visual del jackpot). Nos comprometemos a completar la robustez que quedó a medias (validar color CSS y rechazar repetidos) y a alcanzar las pruebas compartidas del curso con anticipación, en vez de dejarlas para el final.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

En este ciclo la práctica XP más útil fue el diseño incremental / refactorización. Tener el código de los ciclos anteriores bien separado (la lógica funcionando en modo invisible, la parte visual aparte) fue lo que hizo posible agregar solve y simulate sin romper nada, y usar la máquina como herramienta de prueba. Que la lógica no dependiera de la parte grafica fue justo lo que permitió resolver a ciegas.

8. ¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.

ICPC Foundation. (2025). Problem I: Slot Machine – 49th ICPC World Championship, Baku. ADA University.

Fue la más útil porque de ahí salió la regla clave del algoritmo: que solo se puede consultar el número de símbolos distintos después de cada acción, y que hay un límite de 10 000 acciones. Entender esas reglas fue lo que nos llevó a diseñar solve como un proceso a ciegas y no mirando la configuración.

1. ¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.

Mini-ciclo 14 Tipos de rueda:

Objetivo: ofrecer tres tipos de rueda (normal, lefty, rebel). Convertimos Wheel en una clase abstracta y creamos las subclases NormalWheel, LeftyWheel (copia el estado de la rueda a su izquierda) y RebelWheel (no se deja bloquear, intercambiar ni eliminar). Es la base del ciclo porque introduce el polimorfismo de ruedas del que dependen los demás. Los métodos nuevos son type(), allowsLock()/allowsSwap()/allowsDelete(), setLeftNeighbor(), currentIndex() y la fábrica createWheel() en SlotMachine.

Mini-ciclo 15 Tipos de símbolo:

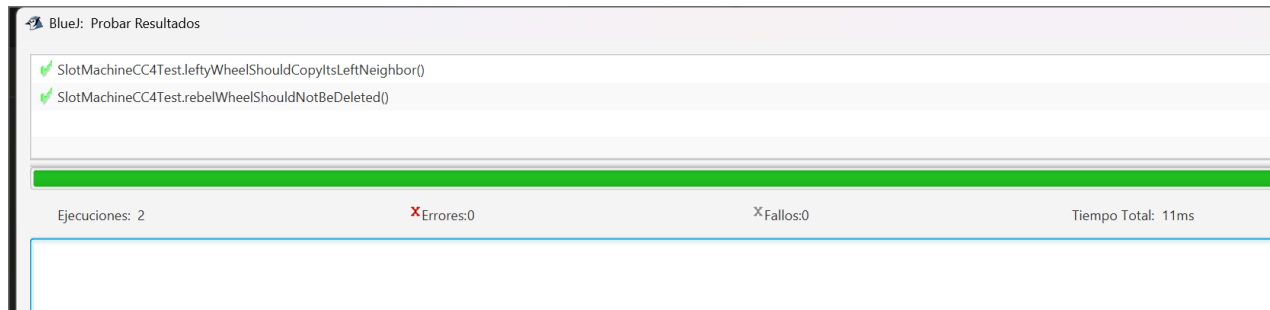
Objetivo: ofrecer tres tipos de símbolo (normal, ephemeral, shy). Convertimos Symbol en abstracta y creamos NormalSymbol, EphemeralSymbol (encoge en cada selección) y ShySymbol (alterna visible/invisible). Para lograrlo agregamos onSelected() en Symbol, el método select() en Wheel y la fábrica estática Symbol.create(). Usa el mismo patrón del mini-ciclo 14, por eso va después.

Mini-ciclo 16 Tipo propio:

Objetivo: cumplir el requisito de proponer un tipo nuevo. Creamos GrowingSymbol, el opuesto de ephemeral: en cada selección crece hasta un máximo. Depende del mini-ciclo 15 porque reutiliza toda la jerarquía de símbolos.

Mini-ciclo 17 Pruebas:

Objetivo: verificar lo construido con SlotMachineC4Test (pruebas propias), las dos pruebas compartidas SlotMachineCC4Test y las pruebas de aceptación. Depende de todos los anteriores, porque no se puede probar lo que aún no existe.



BlueJ: Probar Resultados

- ✔ SlotMachineCC4Test.LeftyWheelShouldCopyItsLeftNeighbor()
- ✔ SlotMachineCC4Test.RebelWheelShouldNotBeDeleted()

Ejecuciones: 2 Errores:0 Fallos:0 Tiempo Total: 11ms

✔ SlotMachineC4Test.shouldCreateEachSymbolType()
✔ SlotMachineC4Test.rebelWheelShouldNotAllowLockSwapOrDelete()
✔ SlotMachineC4Test.normalWheelShouldAllowLockSwapAndDelete()
✔ SlotMachineC4Test.leftyWheelShouldCopyLeftNeighborWhenSpun()
✔ SlotMachineC4Test.shouldNotLockRebelWheel()
✔ SlotMachineC4Test.shouldNotDeleteRebelWheel()
✔ SlotMachineC4Test.leftyWheelWithoutNeighborShouldSpinNormally()
✔ SlotMachineC4Test.shouldReportWheelTypes()
✔ SlotMachineC4Test.unknownSymbolTypeShouldBecomeNormal()
✔ SlotMachineC4Test.specialSymbolsShouldSurviveManySelections()
✔ SlotMachineC4Test.shouldLockNormalWheel()

Ejecuciones: 11	✖ Errores:0	✖ Fallos:0	Tiempo Total: 199ms
-----------------	-------------	------------	---------------------

2. ¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿Por qué?

El proyecto está en su estado más completo hasta ahora. Los mini-ciclos 1 (Esqueleto), 2 (Gestión de elementos), 3 (Dinamismo), 4 (Visual), 6 (Cierre), 7 y 8 (extensiones del Ciclo 2) y 10, 11, 12 (del Ciclo 3) siguen completos. De los nuevos de este ciclo, el 14 (Tipos de rueda), el 15 (Tipos de símbolo) y el 16 (Tipo propio) quedaron completos. El mini-ciclo 17 (Pruebas) quedó completo esta vez: además de las pruebas propias y de aceptación, por fin alcanzamos a hacer las pruebas compartidas del curso, que en los ciclos anteriores siempre nos quedaban pendientes. El mini-ciclo 5 (Robustez) sigue parcial: aún nos falta validar que el color sea realmente un color CSS y rechazar colores repetidos, que lo dejamos planeado para la entrega final. La razón de este estado es que seguimos avanzando de lo que no depende de nada hacía lo que depende de todo, y dejamos las pruebas al final de cada ciclo.

3. ¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Nombre)

Juan Sebastián Garavito Mejia: 16 horas

Laila Zareth Romano Guerrero: 16 horas

4. ¿Cuál consideran fue el mayor logro? ¿Por qué?

Nuestro mayor logro fue aplicar herencia y polimorfismo de verdad: convertir Wheel y Symbol en clases abstractas con sus subclases, logrando que SlotMachine trate a cualquier tipo de rueda o símbolo sin preguntar de qué tipo es. Lo consideramos el mayor logro porque es el salto conceptual más grande del proyecto: pasamos de tener clases únicas a tener jerarquías donde cada tipo decide su propio comportamiento. Ver que podíamos agregar tipos nuevos (como nuestro GrowingSymbol) tocando solo las fábricas, sin modificar la lógica de la máquina, nos confirmó que entendimos para qué sirve el polimorfismo y no solo cómo escribirlo. Además, por primera vez cerramos las pruebas compartidas del curso dentro del mismo ciclo.

5. ¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?

El mayor problema técnico fue cómo hacer que los símbolos ephemeral, shy y growing reaccionaran "al ser seleccionados en la rueda" sin que la clase Wheel tuviera que conocer cada tipo de símbolo. Al principio pensábamos meter if sobre el tipo dentro de Wheel, pero eso rompía la idea del polimorfismo. Lo resolvimos agregando el método onSelected() en la clase Symbol (que cada subtipo redefine a su manera) y un método select() en Wheel que, al terminar un giro, le avisa al símbolo visible que fue seleccionado. Así Wheel solo dice "fuiste seleccionado" y cada símbolo decide qué hacer (normal: nada; ephemeral: encoge; shy: se esconde; growing: crece). También tuvimos que resolver que al volver Symbol y Wheel abstractas ya no se podía hacer new Symbol() ni new Wheel(), lo que nos obligó a crear fábricas (Symbol.create() y createWheel()) para centralizar la creación de los subtipos.

6. ¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?

Como equipo hicimos bien el refactorizar sin romper lo que ya funcionaba: comprobamos que el resolvedor del Ciclo 3 (solve) siguiera funcionando igual después de meter toda la jerarquía de tipos. También mantuvimos el orden de los mini-ciclos y esta vez sí alcanzamos a hacer las pruebas compartidas. Nos comprometemos a completar la robustez que llevamos arrastrando desde el Ciclo 1 (validar color CSS y rechazar colores repetidos) y a corregir los detalles de diseño que nos observó la profesora antes de la entrega final, como ajustar que lock devuelva int según el enunciado.

7. Considerando las prácticas XP incluidas en los laboratorios, ¿cuál fue la más útil? ¿Por qué?

En este ciclo la práctica XP más útil fue la refactorización. Como el ciclo pedía "Refactoring y Extensión", tuvimos que reorganizar clases que ya funcionaban (Wheel y Symbol) para que admitieran variantes, sin cambiar su comportamiento externo. Tener las pruebas de los ciclos anteriores fue justo lo que nos dio confianza para refactorizar: cada vez que convertíamos una clase en abstracta, volvíamos a correr las pruebas y confirmábamos que no habíamos roto nada. Eso nos enseñó que refactorizar sin pruebas sería adivinar.